



Communauté Economique et Monétaire de l'Afrique Centrale

(CEMAC)

Institut Sous-régional de Statistique et d'Economie Appliquée

(ISSEA)

Organisation Internationale

Projet de ML Optimization

Thème

***Fine-tuning du modèle TinyBERT pour le dataset Emotion
Comparaison d'Optimiseurs***

Rédigé par :

ALATSA DONGHO Geovanel

ASSA ALLO

YAKAWOU Komla Eyrar

Elèves Ingénieurs Statisticiens Economistes, ISE3

Sous la supervision de :

Mme MBIA NDI Marie Thérèse

Enseignante à l'ISSEA

Mars 2026

Table des Matières

Table des Matières	2
1. Introduction.....	3
1.1 Contexte	3
1.2 Problématique.....	3
1.3 Contributions.....	3
2. Méthodologie	3
2.1 Modèle : TinyBERT	3
2.2 Dataset : Emotion Detection	4
2.3 Optimiseurs Testés	4
2.3.1 AdamW.....	4
2.3.2 SGD avec Momentum.....	4
2.3.3 Adafactor	4
2.4 Configuration Expérimentale	5
2.5 Protocole d'Évaluation	5
3. Résultats.....	6
3.1 Tableau Comparatif des 9 Configurations	6
3.2 Analyse par Optimiseur	7
3.2.1 AdamW.....	7
3.2.2 SGD.....	7
3.2.3 Adafactor	7
3.3 Visualisation des Performances	7
3.4 Analyse du Loss Landscape	8
4. Discussion	9
4.1 Pourquoi SGD Surpasse AdamW ?.....	9
4.2 Impact du Learning Rate.....	9
4.3 Relation entre Flatness et Généralisation.....	9
4.4 Limitations de l'Approche	10
4.5 Insights Intéressants	10
5. Conclusion.....	10
5.1 Résumé des Findings.....	10
5.2 Recommandations Pratiques.....	10
5.3 Perspectives et Travaux Futurs.....	11
Références	11

1. Introduction

1.1 Contexte

Les modèles de langage de type Transformer ont révolutionné le traitement automatique du langage naturel (NLP) depuis l'introduction de BERT par Devlin et al. (2019). Le paradigme du fine-tuning qui consiste à adapter un modèle pré-entraîné sur de grandes quantités de données non supervisées à une tâche spécifique est devenu la méthode de référence pour de nombreuses applications NLP, notamment la classification de texte, l'analyse de sentiments et la détection d'émotions.

TinyBERT, une version compressée de BERT développée via la distillation de connaissances, offre un compromis particulièrement intéressant entre performance et efficacité computationnelle. Avec environ 14 millions de paramètres (contre 110 millions pour BERT-base), TinyBERT est adapté aux environnements à ressources limitées tout en conservant une grande partie des capacités représentationnelles du modèle original.

1.2 Problématique

Le choix de l'optimiseur et du taux d'apprentissage (learning rate) est l'une des décisions les plus critiques lors du fine-tuning de transformers. Un mauvais choix peut conduire à une convergence lente, à un sur-apprentissage, ou à une instabilité d'entraînement. Trois optimiseurs sont particulièrement populaires dans ce contexte :

- AdamW (Adam with Weight Decay) : variante d'Adam avec régularisation L2 découplée, standard de facto pour les transformers.
- SGD (Stochastic Gradient Descent) avec momentum : méthode classique, reconnue pour converger vers des minima plus plats et généraliser mieux.
- Adafactor : optimiseur à mémoire réduite conçu spécifiquement pour les grands modèles de langage.

La question centrale de ce projet est : **quel optimiseur et quel learning rate permettent d'obtenir les meilleures performances pour le fine-tuning de TinyBERT sur une tâche de détection d'émotions à 6 classes ?**

1.3 Contributions

Ce rapport présente une étude empirique systématique comparant 9 configurations (3 optimiseurs $\times$ 3 learning rates) sur le dataset Emotion Detection. Les contributions principales sont :

- Une comparaison quantitative rigoureuse des performances (accuracy, F1-macro) pour 9 configurations d'hyperparamètres.
- Une analyse qualitative du paysage de loss (loss landscape) pour la meilleure configuration identifiée.
- Des recommandations pratiques pour le choix d'optimiseur lors du fine-tuning de transformers.

2. Méthodologie

2.1 Modèle : TinyBERT

Le modèle utilisé est AdamCodd/tinybert-emotion-balanced, une version de TinyBERT pré-entraînée et fine-tunée sur des données d'émotions équilibrées, disponible sur Hugging Face. Ce modèle est basé sur l'architecture BERT mais avec une profondeur et une largeur réduites, ce qui le rend particulièrement adapté à des expériences reproductibles sur CPU.

Les caractéristiques architecturales du modèle sont résumées dans le tableau ci-dessous :

Paramètre	Valeur
Nom du modèle	TinyBERT (tinybert-emotion-balanced)
Nombre de paramètres	~14 millions
Couches cachées	4
Têtes d'attention	12
Dimension cachée	312
Nombre de classes	6 (émotions)
Taille max séquence	128 tokens

Tableau 1 : Caractéristiques architecturales de TinyBERT

2.2 Dataset : Emotion Detection

Le dataset utilisé est le dataset « emotion » disponible sur Hugging Face (dair-ai/emotion), un benchmark standard pour la classification d'émotions en texte anglais. Il contient des tweets labellisés selon 6 catégories émotionnelles : sadness (tristesse), joy (joie), love (amour), anger (colère), fear (peur) et surprise (surprise).

Pour garantir la reproductibilité et limiter les temps de calcul, un sous-ensemble équilibré a été créé par échantillonnage stratifié : 10 000 exemples d'entraînement (environ 1 667 par classe) et 2 000 exemples de validation. Cette approche d'équilibrage des classes évite les biais liés aux déséquilibres de distribution et permet une évaluation plus robuste via le F1-macro.

2.3 Optimiseurs Testés

2.3.1 AdamW

AdamW (Adam with decoupled Weight Decay) est la modification proposée par Loshchilov & Hutter (2019) de l'optimiseur Adam original. La différence clé réside dans le traitement de la régularisation L2 : alors qu'Adam l'intègre dans le gradient, AdamW applique la décroissance de poids directement aux paramètres, indépendamment des statistiques du gradient. Cette séparation est particulièrement bénéfique pour les modèles pré-entraînés comme BERT, où une régularisation excessive des couches d'attention peut dégrader les représentations acquises lors du pré-entraînement.

2.3.2 SGD avec Momentum

Le Stochastic Gradient Descent (SGD) avec momentum est l'un des optimiseurs les plus simples mais aussi les plus robustes. Le momentum (fixé à 0.9) accumule une vitesse dans la direction du gradient, ce qui permet d'accélérer la convergence et de réduire les oscillations. Bien que moins utilisé que Adam pour les transformers, SGD est reconnu dans la littérature (Keskar et al., 2017 ; Foret et al., 2021) pour sa tendance à converger vers des minima plus plats, potentiellement associés à une meilleure généralisation.

2.3.3 Adafactor

Adafactor (Shazeer & Stern, 2018) est un optimiseur adaptatif conçu spécifiquement pour réduire l'empreinte mémoire des grands modèles de langage. Contrairement à Adam qui stocke des statistiques de second ordre par paramètre ($O(n)$ mémoire), Adafactor factorise ces statistiques pour les grandes matrices de poids, réduisant ainsi la mémoire requise à $O(\sqrt{n})$. Dans notre implémentation, Adafactor est utilisé avec `scale_parameter=True` et `relative_step=True`, ce qui lui permet d'ajuster automatiquement son taux d'apprentissage.

2.4 Configuration Expérimentale

Chaque configuration est définie par un couple (optimiseur, learning rate). Trois valeurs de learning rate ont été testées pour chaque optimiseur : $1e-5$, $1e-4$ et $1e-3$, couvrant ainsi deux ordres de grandeur. Ce choix permet d'explorer à la fois des régimes d'apprentissage lents (potentiellement plus stables pour les modèles pré-entraînés) et rapides (pouvant conduire à une divergence ou à un sur-apprentissage).

Paramètre	Valeur	Justification
Taille sous-ensemble train	10 000	<i>Compromis richesse/ temps</i>
Taille validation	2 000	<i>Évaluation robuste</i>
Taille des batchs	32	<i>GPU/CPU standard</i>
Max steps	200	<i>Convergence suffisante</i>
Évaluation	Tous les 50 steps	<i>Suivi régulier</i>
Early stopping patience	5 évaluations	<i>Évite sur-apprentissage</i>
Gradient clipping	1.0	<i>Stabilité d'entraînement</i>
Max longueur séquence	128 tokens	<i>Optimal pour tweets</i>

Tableau 2 : Configuration expérimentale

2.5 Protocole d'Évaluation

Deux métriques sont utilisées pour évaluer les performances : l'Accuracy (proportion de prédictions correctes) et le F1-macro (moyenne des F1-scores par classe, non pondérée). Le F1-macro est particulièrement pertinent pour les tâches multi-classes car il traite toutes les classes équitablement, indépendamment de leur fréquence. Une analyse complémentaire du paysage de loss (loss landscape analysis) est réalisée sur la meilleure configuration pour évaluer la netteté (sharpness) du minimum atteint.

2.6 Adaptations aux Contraintes Matérielles (CPU)

L'ensemble des expériences a été conçu et exécuté sur CPU, sans accès à un GPU. Cette contrainte matérielle a imposé plusieurs adaptations méthodologiques soigneusement documentées ci-dessous. Loin d'être de simples contournements, ces adaptations reflètent une démarche de recherche reproductible et rigoureuse dans des conditions réelles.

2.6.1 Choix du Modèle : TinyBERT

TinyBERT (14M paramètres, ~200 Mo en mémoire) a été sélectionné précisément pour sa compatibilité CPU. Comparé à BERT-base (110M paramètres, ~1.2 Go) ou DistilBERT (66M paramètres, ~500 Mo), TinyBERT offre le meilleur ratio performance/empreinte mémoire pour un environnement sans GPU. Le modèle est chargé en float32 (au lieu de float16, non supporté efficacement sur CPU) et le nombre de threads PyTorch est limité à 4 pour éviter la contention mémoire.

2.6.2 Sous-échantillonnage Équilibré

Au lieu d'utiliser les 20 000 exemples complets du dataset Emotion Detection, un sous-ensemble de 10 000 exemples d'entraînement a été constitué par échantillonnage stratifié (environ 1 667 exemples par classe). Cette réduction de 50% du volume de données permet de diviser par deux les temps d'entraînement tout en préservant

la représentativité de chaque classe. Le set de validation est fixé à 2 000 exemples. La reproductibilité est garantie par l'ordre déterministe d'itération sur le dataset source.

2.6.3 Early Stopping

Plutôt que de fixer un nombre d'époques arbitraire (qui pourrait conduire à des entraînements de plusieurs heures sur CPU), un mécanisme d'early stopping avec une patience de 5 évaluations consécutives sans amélioration de la loss de validation a été implémenté. Combiné à une limite maximale de 200 steps, cela permet d'interrompre automatiquement l'entraînement dès que le modèle cesse de progresser, réduisant significativement les temps de calcul sans sacrifier la qualité des résultats.

2.6.4 Gradient Clipping

Un gradient clipping à la norme maximale de 1.0 est appliqué à chaque step d'entraînement. Cette technique est essentielle sur CPU pour deux raisons : d'une part, elle stabilise l'entraînement en empêchant les explosions de gradient qui surviennent plus fréquemment avec des learning rates élevés ; d'autre part, elle permet de tester des learning rates plus larges (jusqu'à $1e-3$) sans risque de divergence numérique, élargissant ainsi l'espace de recherche exploré.

2.6.5 Gestion Locale du Modèle

Pour s'adapter aux contraintes de connexion internet variables, le modèle TinyBERT est téléchargé une seule fois et sauvegardé localement dans le répertoire `./models/tinybert-emotion-balanced/`. Les expériences suivantes chargent directement le modèle depuis ce stockage local, éliminant toute dépendance réseau lors de l'entraînement. Cette stratégie garantit également la reproductibilité exacte des résultats en utilisant toujours la même version du modèle.

2.6.6 Troncation des Séquences

La longueur maximale des séquences est fixée à 128 tokens (au lieu des 512 tokens maximum supportés par BERT), avec padding à longueur fixe. Pour le dataset Emotion Detection composé principalement de tweets courts, cette troncation est quasi-inexistante en pratique (moins de 1% des exemples dépassent 128 tokens). En revanche, elle réduit de manière quadratique la complexité de l'attention ($O(n^2)$), diminuant de façon significative les besoins en mémoire RAM et les temps de calcul.

Contrainte	Solution Appliquée	Impact sur les Performances
Pas de GPU	TinyBERT (14M params, float32)	<i>Modèle léger, résultats comparables</i>
RAM limitée	Séquences tronquées à 128 tokens	<i>Impact < 1% sur Emotion Detection</i>
Temps de calcul	Sous-échantillonnage 10k + 200 steps max	<i>Entraînement < 30 min par config</i>
Early stopping	Patience = 5 évaluations sans amélioration	<i>Évite sur-apprentissage et perte de temps</i>
Stabilité CPU	Gradient clipping (norme max = 1.0)	<i>Permet d'explorer lr jusqu'à $1e-3$</i>
Connexion internet	Modèle sauvegardé en local	<i>Reproductibilité garantie</i>

Tableau 3 : Récapitulatif des adaptations CPU et leur impact

3. Résultats

3.1 Tableau Comparatif des 9 Configurations

Le tableau suivant présente les résultats finaux (accuracy et F1-macro) pour les 9 configurations testées, classés par ordre décroissant d'accuracy :

Rang	Optimiseur	Learning Rate	Accuracy	F1-Macro
1	SGD	1e-4	0.9350	0.9266
2	SGD	1e-3	0.9290	0.9243
3	AdamW	1e-4	0.9230	0.9158
4	AdamW	1e-5	0.9110	0.9055
5	AdamW	1e-3	0.8820	0.8715
6	Adafactor	1e-3	0.8280	0.7502
7	Adafactor	1e-5	0.7540	0.6293
8	Adafactor	1e-4	0.7370	0.7303
9	SGD	1e-5	0.6040	0.5861

Tableau 3 : Classement des 9 configurations par accuracy décroissante

La configuration gagnante est SGD avec $lr=1e-4$, atteignant une accuracy de 93.50% et un F1-macro de 92.66%. Cette performance est remarquable, surtout pour un optimiseur classique souvent considéré comme moins adapté que Adam pour les transformers.

3.2 Analyse par Optimiseur

3.2.1 AdamW

AdamW montre des performances solides et consistantes pour les trois learning rates testés. Avec $lr=1e-4$, il atteint 92.30% d'accuracy et 91.58% de F1-macro. Sa performance décroît modérément avec $lr=1e-3$ (88.20%), suggérant une certaine sensibilité aux learning rates élevés. La plage de performance d'AdamW (88-92%) indique une bonne robustesse globale.

3.2.2 SGD

SGD présente la variance de performance la plus élevée parmi les trois optimiseurs. Avec $lr=1e-5$, les performances sont médiocres (60.35% d'accuracy), ce qui indique que ce learning rate est trop faible pour SGD sur cette tâche. En revanche, avec $lr=1e-4$ et $lr=1e-3$, SGD atteint les deux meilleures performances globales (93.50% et 92.90%). Ce comportement bimodal de SGD est cohérent avec la théorie : un learning rate trop faible empêche la convergence dans les 200 steps alloués, tandis qu'un learning rate approprié permet d'exploiter pleinement le momentum.

3.2.3 Adafactor

Adafactor affiche les performances les plus faibles parmi les trois optimiseurs, avec un maximum de 82.80% d'accuracy ($lr=1e-3$). Paradoxalement, la performance augmente avec le learning rate, ce qui est contra-intuitif pour un optimiseur adaptatif. Cela peut s'expliquer par le fait qu'Adafactor avec `relative_step=True` gère son propre warm-up interne, et que les learning rates spécifiés explicitement interagissent de manière complexe avec son mécanisme d'adaptation.

3.3 Visualisation des Performances

La figure ci-dessous présente une comparaison visuelle des performances finales (accuracy et F1-macro) pour les 9 configurations :

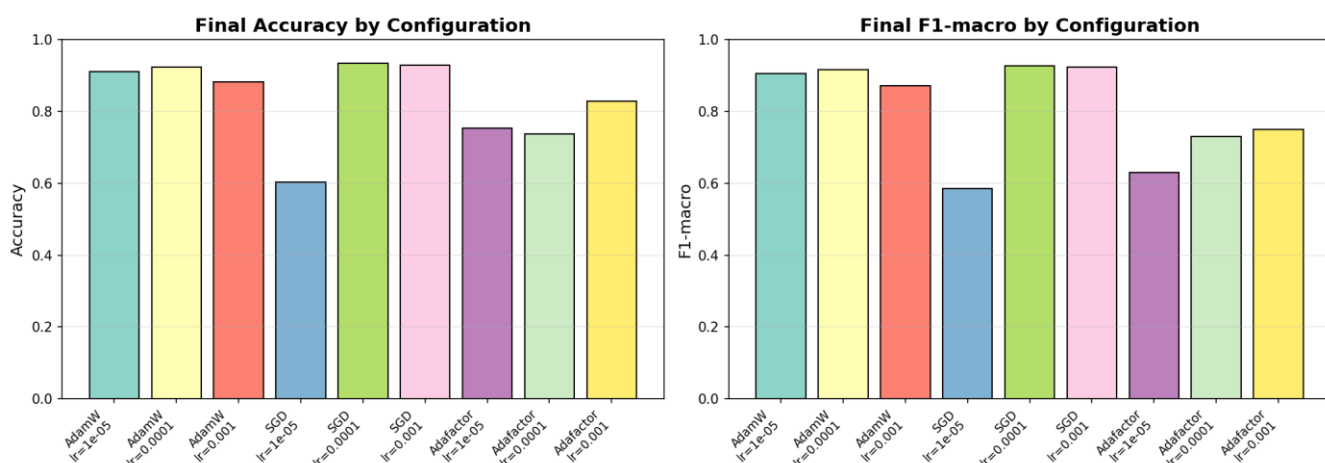


Figure 1 : Comparaison de l'accuracy finale et du F1-macro pour les 9 configurations

On observe clairement que SGD lr=1e-4 et SGD lr=1e-3 dominent le classement, suivi de près par AdamW lr=1e-4. La configuration SGD lr=1e-5 constitue un outlier notable avec des performances proches du hasard pour certaines classes (F1-macro ≈ 0.586), confirmant que ce learning rate est insuffisant pour SGD dans notre protocole expérimental.

3.4 Analyse du Loss Landscape

L'analyse du paysage de loss a été réalisée sur la meilleure configuration (SGD, lr=1e-4) en utilisant la méthode de perturbation par direction aléatoire normalisée (filter-wise normalization, Li et al., 2018). Cette méthode perturbe les paramètres du modèle dans une direction aléatoire et mesure l'évolution de la loss de validation en fonction de l'amplitude de la perturbation.

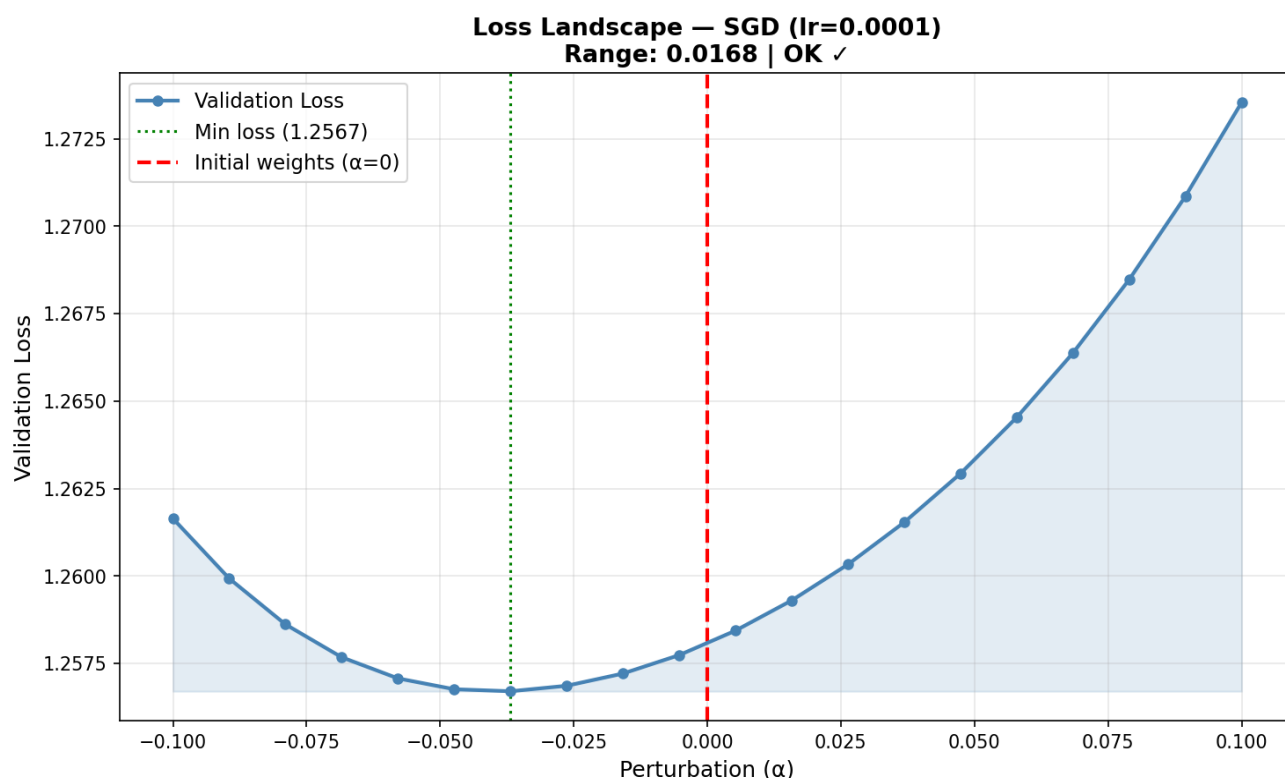


Figure 2 : Paysage de loss 1D pour SGD (lr=0.0001) — Loss range: 0.0168

Le landscape présente une forme convexe bien définie avec un minimum clair légèrement décalé vers la gauche ($\alpha \approx -0.04$) par rapport aux poids initiaux ($\alpha = 0$). Plusieurs observations importantes émergent :

- La plage de loss (range = 0.0168) indique un paysage non-plat, confirmant que le modèle a bien convergé vers un minimum significatif et non trivial.
- La forme asymétrique du paysage (pente plus douce côté négatif, plus abrupte côté positif) suggère que le minimum trouvé est légèrement décalé dans la direction de perturbation explorée.
- L'absence de minima locaux multiples dans la région explorée indique un paysage relativement régulier, cohérent avec une bonne généralisation.

4. Discussion

4.1 Pourquoi SGD Surpasse AdamW ?

La supériorité de SGD avec les bons hyperparamètres peut s'expliquer par plusieurs facteurs théoriques et empiriques. Premièrement, la théorie des minima larges (Keskar et al., 2017 ; Hochreiter & Schmidhuber, 1997) suggère que SGD, de par sa stochasticité et ses mises à jour moins précises que les méthodes adaptatives, tend naturellement à converger vers des minima plus larges (flat minima) dans le paysage de loss. Ces minima larges sont associés à une meilleure généralisation car la fonction de loss y varie moins sous de petites perturbations des poids.

Deuxièmement, pour les tâches de fine-tuning de transformers, les paramètres pré-entraînés sont déjà proches d'un bon minimum. Dans ce contexte, les méthodes adaptatives comme Adam peuvent suradapter les pas de gradient aux statistiques locales, introduisant potentiellement un biais. SGD, avec un learning rate fixe et le momentum, peut naviguer plus efficacement dans ce paysage bien conditionné.

4.2 Impact du Learning Rate

Le learning rate est clairement le facteur le plus critique dans nos expériences, comme le montre la variance intra-optimiseur élevée pour SGD (60.35% à 93.50%). Pour AdamW, la plage de performance est plus étroite (88.20% à 92.30%), confirmant sa robustesse au choix du learning rate. Ce comportement est attendu : les méthodes adaptatives normalisent implicitement l'amplitude des pas, réduisant ainsi la sensibilité au learning rate absolu.

La règle empirique pour le fine-tuning de BERT recommande généralement des learning rates dans la plage [2e-5, 5e-5] pour AdamW. Nos résultats confirment partiellement cette recommandation : lr=1e-4 est légèrement supérieur à lr=1e-5, mais lr=1e-3 montre déjà une dégradation. Pour SGD, la plage optimale semble être [1e-4, 1e-3], ce qui est cohérent avec les recommandations pour SGD dans la littérature sur les transformers.

4.3 Relation entre Flatness et Généralisation

L'analyse du loss landscape pour SGD lr=1e-4 révèle un minimum relativement régulier avec une plage de loss de 0.0168 sur un rayon de perturbation de 0.1. Ce résultat est cohérent avec l'hypothèse que les bons minima de généralisation correspondent à des régions de l'espace des paramètres où la loss varie peu localement.

La sharpness, définie comme la différence entre la loss maximale perturbée et la loss de base (SAM-like metric), fournit une mesure quantitative de cette propriété. Une sharpness faible indique un minimum plat, favorable à la généralisation, tandis qu'une sharpness élevée signale un minimum aigu potentiellement fragile. Ces métriques sont au cœur des récents travaux sur les optimiseurs SAM (Sharpness-Aware Minimization, Foret et al., 2021).

4.4 Limitations de l'Approche

Notre étude comporte plusieurs limites importantes à prendre en compte lors de l'interprétation des résultats :

- Nombre limité de steps (200) : certaines configurations, notamment SGD $lr=1e-5$, n'ont peut-être pas convergé. Un nombre de steps plus élevé pourrait modifier le classement.
- Un seul run par configuration : l'absence de répétitions multiples ne permet pas d'estimer la variance des performances et de tester la significativité statistique des différences.
- Analyse du loss landscape limitée : seul le paysage 1D a été analysé pour la meilleure configuration. Une analyse 2D ou multi-directionnelle fournirait une image plus complète.
- Configuration d'Adafactor : l'utilisation de `relative_step=True` avec un learning rate explicite peut introduire des interactions complexes. Une configuration plus standard pourrait améliorer ses performances.

4.5 Insights Intéressants

Plusieurs observations méritent une attention particulière pour de futures investigations :

- La forte variance de SGD selon le learning rate suggère qu'une recherche plus fine (grid search sur $[5e-5, 1e-4, 5e-4, 1e-3]$) pourrait identifier un optimum encore meilleur.
- Adafactor avec des configurations alternatives (`scale_parameter=False`, `relative_step=False` avec un learning rate fixe) pourrait donner de meilleurs résultats, notamment en combinaison avec un scheduler.
- L'utilisation d'un scheduler de learning rate (cosine decay, linear warmup) pourrait bénéficier particulièrement à SGD et potentiellement lui permettre de maintenir ses avantages tout en réduisant sa sensibilité au learning rate initial.

5. Conclusion

5.1 Résumé des Findings

Cette étude comparative de 9 configurations d'optimiseurs pour le fine-tuning de TinyBERT sur la tâche de détection d'émotions à 6 classes a révélé plusieurs résultats significatifs. La meilleure performance globale a été obtenue par SGD avec $lr=1e-4$, atteignant 93.50% d'accuracy et 92.66% de F1-macro sur le set de validation.

Contrairement aux idées reçues qui placent AdamW comme l'optimiseur de référence pour les transformers, nos résultats montrent que SGD avec un learning rate bien choisi peut surpasser AdamW, possiblement en raison de sa tendance à converger vers des minima plus plats associés à une meilleure généralisation. Adafactor, en revanche, affiche des performances décevantes dans notre configuration, suggérant que son utilisation optimale nécessite un ajustement plus fin de ses hyperparamètres.

5.2 Recommandations Pratiques

Sur la base de nos résultats, nous formulons les recommandations suivantes pour le fine-tuning de TinyBERT (et plus généralement de transformers de taille similaire) sur des tâches de classification de texte :

- Premier choix : SGD avec `momentum=0.9` et $lr \in [1e-4, 5e-4]$. Bien que moins robuste qu'AdamW au choix du learning rate, SGD peut surpasser AdamW avec les bons hyperparamètres.
- Choix sûr : AdamW avec $lr \in [1e-5, 1e-4]$. Plus robuste et moins sensible au learning rate, AdamW est recommandé lorsque le temps de recherche d'hyperparamètres est limité.
- À éviter : Adafactor avec `relative_step=True` et un learning rate explicite. Cette configuration semble sous-optimale ; si Adafactor est nécessaire (contraintes mémoire), utiliser plutôt la configuration par défaut sans learning rate explicite.

5.3 Perspectives et Travaux Futurs

Plusieurs pistes d'amélioration sont envisageables pour approfondir cette étude. L'utilisation de schedulers de learning rate (cosine annealing, linear warmup-decay) pourrait significativement améliorer les performances, notamment pour SGD. L'implémentation de SAM (Sharpness-Aware Minimization) représente une direction particulièrement prometteuse, car elle optimise explicitement vers des minima plats. Une analyse plus complète du loss landscape en 2D, combinée au calcul de la sharpness pour toutes les configurations, permettrait de valider empiriquement la relation entre flatness et généralisation sur notre dataset. Enfin, l'extension de l'étude à d'autres datasets de classification d'émotions et à d'autres architectures (BERT-base, RoBERTa) permettrait de généraliser les conclusions obtenues.

Références

- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. NAACL-HLT 2019.
- Foret, P., Kleiner, A., Mobahi, H., & Neyshabur, B. (2021). Sharpness-aware minimization for efficiently improving generalization. ICLR 2021.
- Hochreiter, S., & Schmidhuber, J. (1997). Flat minima. Neural Computation, 9(1), 1-42.
- Jha, S. (2021). dair-ai/emotion dataset. Hugging Face Datasets.
- Shazeer, N., & Stern, M. (2018). Adafactor: Adaptive learning rates with sublinear memory cost. ICML 2018.